



Security Audit

Gala (Galachain SDK)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	46
Conclusion	47
Our Methodology	48
Disclaimers	50
About Hashlock	51

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Gala team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Gala is building a decentralized entertainment ecosystem that spans gaming, music, film, and blockchain technology. Their platform, GalaChain, is an L1 blockchain optimized for entertainment, offering fast transaction speeds, scalability, and energy efficiency. Gala provides developers with tools like Gala Creators and GalaChain Connect to create and deploy Web3 applications seamlessly. The ecosystem supports various projects, including games like Champions Arena and Spider Tanks, Gala Music for blockchain-based music distribution, and Gala Film for decentralized cinema experiences. Gala's mission is to empower creators and players by providing them with real ownership and control over their digital assets.

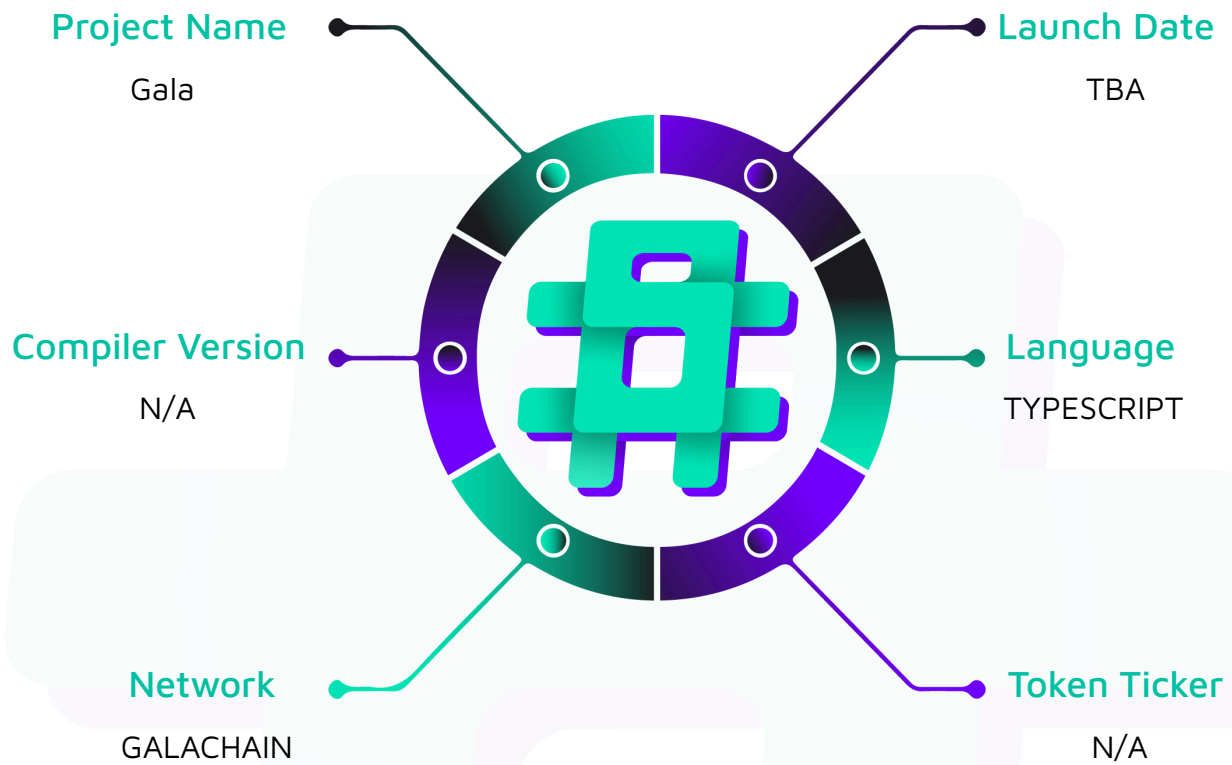
Project Name: Gala

Project Type: Wallet

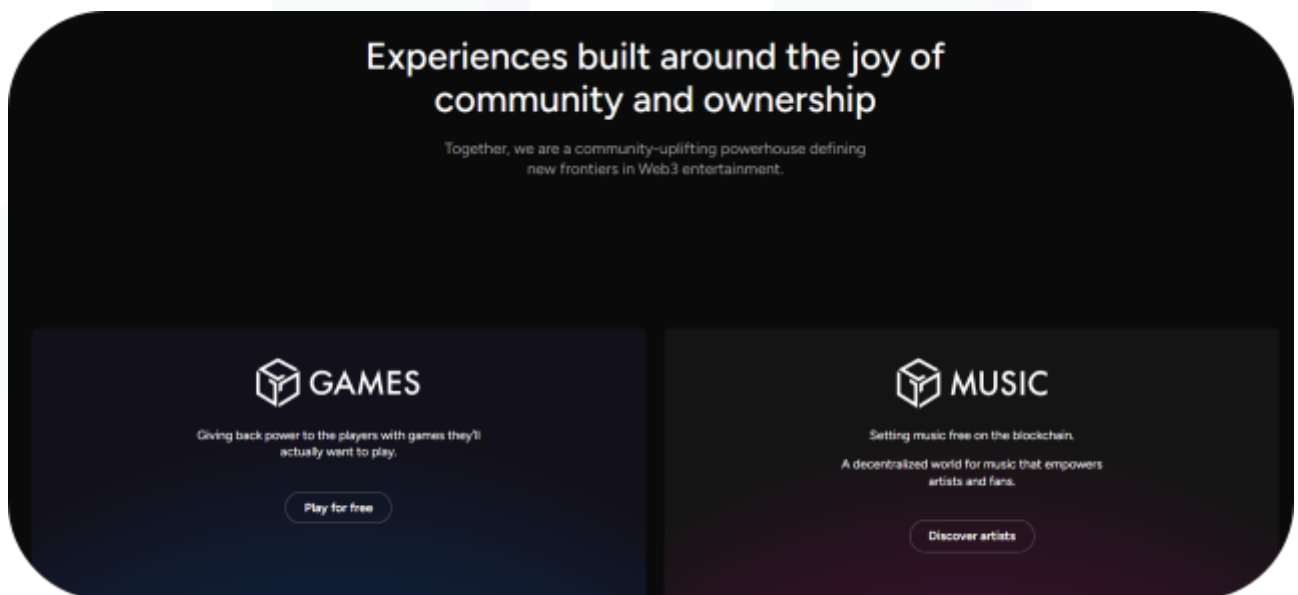
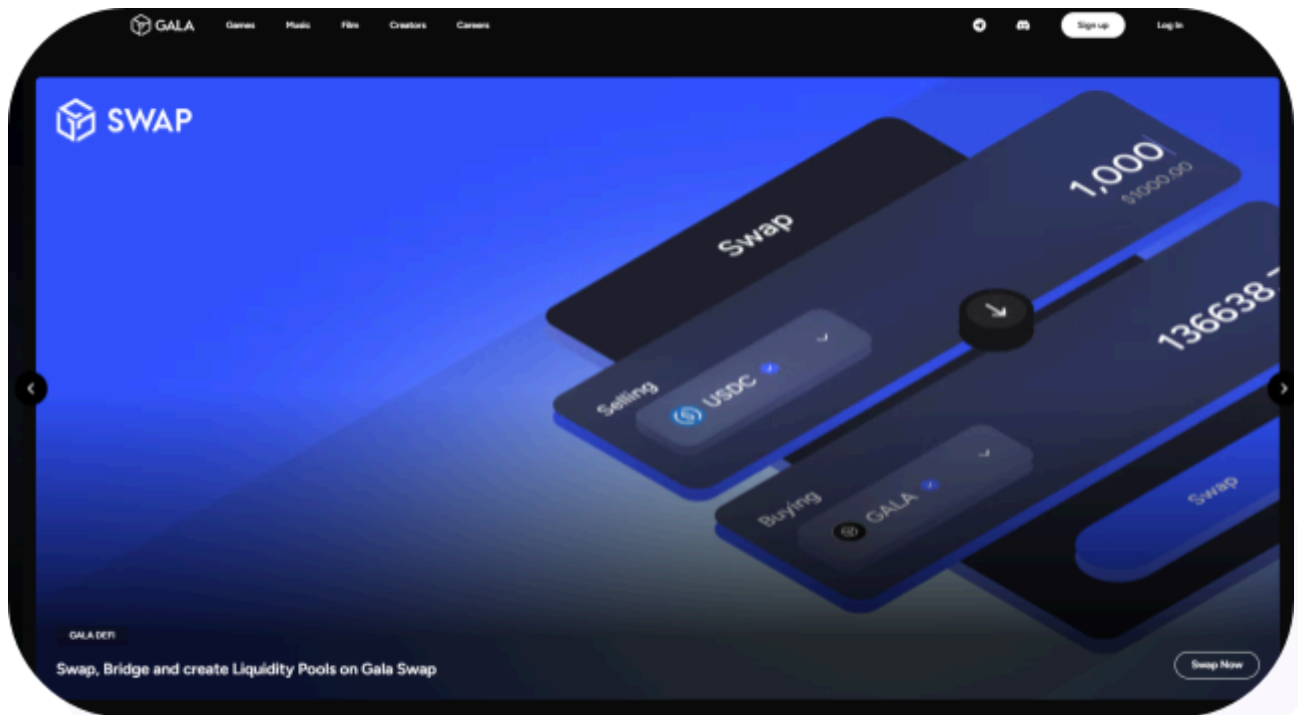
Website: gala.com

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the Rust code within the Gala project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Gala Protocol Smart Contracts
Platform	Galachain / Typescript
Audit Date	November, 2025
Github repo	https://github.com/GalaChain/sdk
Audited GitHub Commit Hash	eb6050f7d8ecdbf9b6b96ebc90216d0eb8068185
Fix Review GitHub Commit Hash	65527727135d03ffcb33ea02999079b3ff7ae2b7

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

7 High severity vulnerabilities

6 Medium severity vulnerabilities

5 Low severity vulnerabilities

3 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
chain-cli - Provides typed CLI commands for managing GalaChain networks. - Handles config loading, environment resolution, and credential selection. - Orchestrates client creation then dispatches contract method invocations.	SDK achieves this functionality.
chain-client - Exposes high-level client API for GalaChain contracts. - Manages identities, signing requests, and network connectivity lifecycle. - Encodes DTOs, sends queries, parses structured responses.	SDK achieves this functionality.
chain-connect - Bridges browser, backend, and wallets to GalaChain client. - Implements pluggable signers for EVM-style wallet providers. - Normalizes provider events and exposes reactive connection state.	SDK achieves this functionality.
chaincode - Implements on-chain business logic for tokens and loans. - Defines transaction functions, validation rules, and authorization checks. - Persists domain entities on ledger using	SDK achieves this functionality.

deterministic schemas.	
chain-api - Exposes HTTP endpoints mapping directly to chaincode. - Serializes request DTOs, validates payloads, enforces schemas. - Wraps responses in GalaChainResponse, standardizing error handling.	SDK achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Gala project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Gala project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] chaincode/src/mint/mintToken.ts#mintToken - Missing validation of `applicableAllowanceKey.grantedTo` allows theft of allowance

Description

The `mintToken` function fails to validate that the `applicableAllowanceKey.grantedTo` parameter matches the actual caller or authorized user.

```
if (applicableAllowanceKey) {  
  
  const allowance: TokenAllowance = await getObjectByKey(  
  
    ctx,  
  
    TokenAllowance,  
  
    ChainObject.getCompositeKeyFromParts(TokenAllowance.INDEX_KEY, [  
  
      applicableAllowanceKey.grantedTo,  
  
      applicableAllowanceKey.collection,  
  
      applicableAllowanceKey.category,  
  
      applicableAllowanceKey.type,  
  
      applicableAllowanceKey.additionalKey,  
  
      applicableAllowanceKey.instance.toString(),  
  
      applicableAllowanceKey.allowanceType.toString(),  
  
      applicableAllowanceKey.grantedBy,  
  
      applicableAllowanceKey.created.toString()  
  
    ])  
  
  );  
  
  applicableAllowanceResponse = [allowance];  
}
```

```
}
```

An attacker can exploit this by:

1. Specifying an arbitrary `applicableAllowanceKey` belonging to another user via the deprecated `MintTokenDto` entry point.
2. Set `dto.owner` to their own address.
3. Forcing the consumption of another user's mint allowance while receiving the minted tokens themselves.

The function later uses this allowance in `useAllowances()` at line 142, which updates the allowance spending for the `grantedTo` user (the victim). In the meantime, the minted tokens are sent to the `owner` parameter (the attacker), resulting in a loss of funds.

Impact

Loss of funds by impersonating other users' allowance.

Recommendation

Consider adding validation to ensure that `applicableAllowanceKey.grantedTo` equals `callingOnBehalf`.

Status

Resolved

[H-02] chaincode/src/allowances/grantAllowance.ts#grantAllowance

Fungible token mint allowance persists after authority removal

Description

When a token authority has granted mint allowances and subsequently has its authority status revoked via `updateTokenClass()` (see `chain-api/src/types/TokenClass.ts:264 ~ 267`), its existing mint allowances remain valid and usable. This is problematic because the mint allowance should fail for users who are not the token's authority.

This works because the ownership check only occurs at allowance creation time:

```
if (allowanceType === AllowanceType.Mint) {  
    // Check if the caller is an authority on the token  
    if (!tokenClass.authorities.includes(ctx.callingUser)) {  
        throw new NotATokenAuthorityError(  
            ctx.callingUser,  
            tokenClass.getCompositeKey(),  
            tokenClass.authorities); }  
}
```

Additionally, the `useAllowances` function in `chaincode/src/allowances/useAllowances.ts :44` does not validate that the user still has the mint token's authority before allowing the mint.

Impact

Revoked authorities can continue minting tokens indefinitely after their authority is removed.

Recommendation

Consider updating the `useAllowances` and `validateMintRequest` functions to re-validate the token's authority before consuming the grant allowance.

Status

Resolved

[H-03] chaincode/src/allowances/grantAllowance.ts#grantAllowance - NFT allowances persist after transfer

Description

When a user grants an allowance for a non-fungible token (NFT) and subsequently transfers that NFT to another user, the original allowance remains valid and usable.

This is because the ownership check only occurs at allowance creation time:

```
// Check that the caller owns the token that they are granting an allowance for:

if (tokenInstance.isNonFungible) {

    // if the token is non-fungible, then we can check owner on tokenInstance

    if (tokenInstance.owner !== ctx.callingUser) {

        throw new InvalidTokenOwnerError(

            ctx.callingUser,

            instanceKey.toStringKey(),

            AllowanceType[allowanceType],

            tokenInstance.owner

        );

    }

}
```

Since there is no mechanism to invalidate allowances when NFT ownership changes, if the NFT is transferred to other users (see chaincode/src/transfer/transferToken.ts:96), the previous owner can still control it.

Impact

This issue allows the previous owner to retain control over the NFT. An attacker may abuse this for profit via the following steps:

1. The attacker grants themselves an allowance on their NFT.
2. The attacker sells the NFT to a victim at a price below market value.
3. The attacker uses the allowance to take back the NFT.

4. The attacker repeats this step to sell their NFTs to other victims.

Recommendation

Consider updating the `useAllowances` and `validateMintRequest` functions to re-validate the NFT's owner before consuming the grant allowance.

Status

Resolved

[H-04]**chaincode/src/allowances/verifyAndUseAllowances.ts#verifyAndUseAllowances** - Duplicate allowances enable double-spending**Description**

The `verifyAndUseAllowances` function accepts an array of allowance keys (`useAllowancesArr`) but does not validate for duplicates.

```
export async function verifyAndUseAllowances(  
  ctx: GalaChainContext,  
  grantedBy: string,  
  tokenInstanceKey: TokenInstanceKey,  
  quantity: BigNumber,  
  tokenInstance: TokenInstance,  
  authorizedOnBehalf: string,  
  actionType: AllowanceType,  
  useAllowancesArr: Array<string>  
) : Promise<boolean> {  
  let applicableAllowances: TokenAllowance[];  
  
  if (useAllowancesArr.length) {  
    const fetchedAllowances = await getObjectsByKeys(ctx, TokenAllowance,  
useAllowancesArr);  
  
    applicableAllowances = fetchedAllowances.filter(  
      (a) =>  
        a.allowanceType === actionType &&  
        a.collection === tokenInstance.collection &&  
        a.category === tokenInstance.category &&  
        a.type === tokenInstance.type &&
```



```
        a.additionalKey === tokenInstance.additionalKey
    );

    applicableAllowances.forEach((allowance) => {
        if (allowance.grantedBy !== grantedBy) {
            throw new AllowanceUsersMismatchError(allowance, grantedBy, authorizedOnBehalf);
        } else if (allowance.grantedTo !== authorizedOnBehalf) {
            throw new AllowanceUsersMismatchError(allowance, grantedBy, authorizedOnBehalf);
        }
    });
}
```

This is problematic because an attacker can provide the same allowance key multiple times, causing the allowance to be counted multiple times in the total allowed quantity.

When `checkAllowances` is called, it sums up the quantities/uses from all allowances in the array, including duplicates. This allows an attacker to artificially inflate their allowance by repeating the same allowance key.

Impact

This vulnerability allows multiplication of allowance spending power, allowing users to transfer, use, or lock tokens beyond their actual allowance:

- `lockToken` in `chaincode/src/locks/lockTokens.ts:117`
- `transferToken` in `chaincode/src/transfer/transferToken.ts:77`
- `useToken` in `chaincode/src/use/useToken.ts:80`

Recommendation

Consider adding duplicate validation at the DTO level using the `@ArrayUnique()` decorator, or manually deduplicating the allowances at runtime.

Status

Resolved

[H-05] chaincode/src/mint/validateMintRequest.ts#validateMintRequest -

Missing caller validation allows allowance bypass

Description

The `validateMintRequest` function fails to validate that the fetched allowance's `applicableAllowanceKey.grantedTo` field matches the actual caller (`callingOnBehalf`).

Specifically, when an `allowanceKey` is provided, the function directly fetches the allowance using the key without verifying that it belongs to the caller:

```
if (params.allowanceKey) {  
  
  const applicableAllowanceKey = params.allowanceKey;  
  
  const allowance: TokenAllowance = await getObjectByKey(  
  
    ctx,  
  
    TokenAllowance,  
  
    ChainObject.getCompositeKeyFromParts(TokenAllowance.INDEX_KEY, [  
  
      applicableAllowanceKey.grantedTo,  
  
      applicableAllowanceKey.collection,  
  
      applicableAllowanceKey.category,  
  
      applicableAllowanceKey.type,  
  
      applicableAllowanceKey.additionalKey,  
  
      applicableAllowanceKey.instance.toString(),  
  
      applicableAllowanceKey.allowanceType.toString(),  
  
      applicableAllowanceKey.grantedBy,  
  
      applicableAllowanceKey.created.toString()  
  
    ])  
  
  );  
  
  results = [allowance];  
  
}
```

Impact

This vulnerability enables unauthorized use of the mint allowance. An attacker can specify another user's `allowanceKey` to use their mint allowance without authorization.

Recommendation

Consider adding validation to ensure the fetched allowance belongs to the caller.

Status

Resolved

[H-06] chain-api/src/types/sale.ts#TokenSaleFulfillment - Composite-key namespace collision enables state confusion and overwrite

Description

Two different chain objects declare the same static index prefix:

- `TokenSaleFulfillment` - `public static INDEX_KEY = "GCTSF"` (class keys: `tokenSaleId`, `fulfilledBy`, `created`).
- `TokenSwapFill` - `public static INDEX_KEY = "GCTSF"` (class keys: `swapRequestId`, `filledBy`, `created`).

Because composite keys are constructed from `INDEX_KEY` + ordered key parts, the resulting ledger keys for a sale fulfillment and a swap fill can become identical whenever `swapRequestId == tokenSaleId`, `filledBy == fulfilledBy`, and `created` match, causing writes or reads to step on each other and returning the wrong object type during scans or lookups.

Impact

A swap fill can be overwritten or be fetched as a sale fulfillment, or vice versa. Additionally, the business logic that enumerates "fulfillments" by prefix may accidentally count swap fills, leading to incorrect balances, fulfillment counts, or refunds.

Recommendation

Assign unique, purpose-specific prefixes, like `TokenSaleFulfillment.INDEX_KEY = "GCTTSF"` and `TokenSwapFill.INDEX_KEY = "GCTSWF"`. Add a unit test to assert *all* `INDEX_KEY` values are unique across the codebase and a data migration to relabel existing keys if any were already written.

Status

Resolved

[H-07] chain-api/src/types/bridge.ts#RequestTokenBridgeOutDto

Bridge-out recipient is an unvalidated string

Description

The `RequestTokenBridgeOutDto.recipient` is only constrained by `@IsString()` and `@IsNotEmpty()`, with no chain-specific address validation. At the same time, `destinationChainId` deliberately supports EVM (Ethereum), TON and Solana.

Accepting arbitrary strings here allows malformed or wrong-chain addresses to be recorded and acted upon by the bridge.

Impact

Tokens can be burned or locked on GalaChain but never claimable on the destination chain due to an invalid or mismatched address. Additionally, the downstream bridge components may hard-fail or go into manual remediation paths when asked to deliver to invalid destinations.

Recommendation

Enforce per-chain recipient validation in the DTO (and defensively in handlers):

- Ethereum: require 0x-prefixed 20-byte addresses and verify EIP-55 checksum.
- TON: validate bounceable and non-bounceable format, network tag and CRC.
- Solana: base58 decoding to a 32-byte ed25519 public key

Reject addresses that do not match the selected `destinationChainId`. Consider normalizing cases and adding checksum verification errors that are explicit for users.

Status

Resolved

Medium

[M-01] chaincode/src/allowances/checkAllowances.ts#isAllowanceSpent -

Incorrect allowance spent validation logic

Description

The `isAllowanceSpent` function uses an AND condition to check if both `usesSpent` and `quantitySpent` are defined before determining if an allowance is spent. This logic is flawed because an allowance should be considered spent if either `uses` or `quantity` is exhausted:

```
function isAllowanceSpent(allowance: TokenAllowance): boolean {  
  
  if (allowance.usesSpent !== undefined && allowance.quantitySpent !== undefined) {  
  
    return (  
  
      allowance.usesSpent.isGreaterThanOrEqualTo(allowance.uses) ||  
  
      allowance.quantitySpent.isGreaterThanOrEqualTo(allowance.quantity)  
  
    );  
  
  }  
  
  return false;  
  
}
```

For example, if `usesSpent` is defined but `quantitySpent` is undefined, the function returns false even if all uses are spent. If `quantitySpent` is defined but `usesSpent` is undefined, the function returns false even if all the quantity is spent.

Impact

Functions that check if allowances are still valid may incorrectly consider spent/consumed allowances as still active.

Recommendation

Consider checking `usesSpent` and `quantitySpent` independently. Examples can be found in `chaincode/src/allowances/fullAllowanceCheck.ts:101~103` and `chaincode/src/allowances/grantAllowance.ts:624~629`.

Status

Resolved

[M-02] chain-api/src/types/TokenBalance.ts#getLockedVestingQuantity -

Rounding error could zero out vested tokens

Description

The `getLockedVestingQuantity` function calculates vested tokens by dividing the total quantity by the vesting period duration (in milliseconds), then multiplying by elapsed time. If the total token amount is less than the vesting period in milliseconds, the division results in 0, leaving all tokens locked indefinitely.

Impact

The `getLockedVestingQuantity` function may return a larger locked token amount than intended.

Recommendation

Consider performing multiplication before division to preserve precision.

Status

Resolved

[M-03] chain-connect/src/chainApis/FeeApi.ts#SetFeeProperties - Incorrect method name and missing signature

Description

The `SetFeeProperties` function contains two issues that prevent it from functioning correctly.

Firstly, the method name is set incorrectly. It calls `"FetchFeeProperties"` instead of `"SetFeeProperties"`.

Secondly, it uses `sign: false` when it should use `sign: true` when updating the fee properties state.

```
public SetFeeProperties(dto: SetFeePropertiesRequest) {  
  
    return this.connection.submit({  
  
        method: "FetchFeeProperties",  
  
        payload: dto,  
  
        sign: false,  
  
        url: this.chainCodeUrl,  
  
        requestConstructor: FeePropertiesDto,  
  
        responseConstructor: FeeProperties  
  
    }); }  

```

Impact

The `SetFeeProperties` function will not work as intended. Users attempting to set fee properties will not receive an error, but will also not achieve the intended fee changes.

Recommendation

Consider updating the method name to `SetFeeProperties` and use `sign: true`.

Status

Resolved

[M-04] chain-api/src/types/dtos.ts#UpdateQuorumDto - Missing minimum and integer validation allows insecure quorum values

Description

The `UpdateQuorumDto.quorum` is only `@IsNotEmpty()` - there is no `@IsInt()` or `@Min(1)`. This permits `0`, negatives, or non-integers to pass DTO validation, undermining multi-signature or threshold semantics if the handler does not add its own checks.

```
export class UpdateQuorumDto extends SubmitCallDTO {  
  
  @JSONSchema({  
  
    description: "New quorum for the user."  
  
  })  
  
  @IsNotEmpty()  
  
  quorum: number;  
  
}
```

Impact

A quorum of `0` effectively disables signature requirements if not re-validated, enabling unauthorized updates to keys and roles via misconfiguration or malicious requests that pass DTO validation.

Recommendation

Enforce `@IsInt()` and `@Min(1)` on `UpdateQuorumDto.quorum`. Optionally cap the quorum at the number of registered public keys to prevent unreachable thresholds.

Status

Resolved

[M-05] chain-cli/src/galachain-utils - Command injection via unquoted and unsafe shell construction

Description

Multiple shell invocations interpolate unquoted variables into command strings and then pass them to a shell (execSync/execSyncStdio). The most critical ones:

1. Key generation (directories and redirections):

```
// create the keys directory
execSync(`mkdir -p ${keysPath}`);
execSync(`mkdir -p ${privateKeysPath}`);

// create the public and private keys files
execSync(`echo '${adminPublicKey}' > ${keysPath}/${DEFAULT_ADMIN_PRIVATE_KEY_NAME}.pub`);
execSync(`echo '${devPublicKey}' > ${keysPath}/${DEFAULT_DEV_PRIVATE_KEY_NAME}.pub`);
execSync(`echo                '${adminPrivateKey.toString()}'                >
${privateKeysPath}/${DEFAULT_ADMIN_PRIVATE_KEY_NAME}`);
execSync(`echo                '${devPrivateKey.toString()}'                >
${privateKeysPath}/${DEFAULT_DEV_PRIVATE_KEY_NAME}`);
```

The `keysPath`, `privateKeysPath` and file names are unquoted - word splitting and command substitution are possible if a crafted path or value with `;`, backticks, `$()`, spaces or quotes is fed in, for example via project path or working dir.

2. Docker inspection & run (image tag):

```
export function getChaincodeImageInfo(imageTag: string): {
  contracts: { contractName: string }[];
  imageArchitecture: string;
  imageSha256: string;
} {
  const inspectCommand = `docker inspect ${imageTag}`;
  let dockerJson;
```

```

try {
    const dockerImageInspect = execSync(inspectCommand);
    dockerJson = JSON.parse(dockerImageInspect);
} catch (e) {
    throw new Error(`Invalid output of 'docker inspect' command:\n\n
    ${inspectCommand}\n\nError: ${e}`);
}

const imageArchitecture = dockerJson?.[0]?.Os + "/" + dockerJson?.[0]?.Architecture;
const imageSha256 = dockerJson?.[0]?.Id;
if (imageArchitecture !== ExpectedImageArchitecture) {
    throw new Error(
        `Unsupported architecture ${imageArchitecture} of Docker image ${imageTag}, ` +
        `expected ${ExpectedImageArchitecture}.`
    );
}

const getContractNamesCommand = `docker run --rm ${imageTag} lib/src/cli.js
get-contract-names | tail -n 1`;

let response = "<command-not-executed-yet>";

```

The `imageTag` is interpolated without quoting. A crafted value can inject options or additional commands.

3. Watch-mode chaincode start (argument escaping):

```

function startChaincodeInWatchMode(fabloRoot: string, args: SingleArg[]): void {
    const chaincodeName = args[0].chaincodeName;

    const commands = [`cd "${fabloRoot}"`, `./chaincode-dev-start.sh "watch"
"${chaincodeName}"`];

    execSyncStdio(commands.join(" && "));
}

```

Although the variable is within double quotes, if `chaincodeName` contains `"` it can break out of quoting - backticks and `$()` also execute within double quotes in most shells.

Impact

An attacker who controls CLI inputs can execute arbitrary OS commands under the CLI's privileges.

Recommendation

Never pass untrusted data to a shell. Use `child_process.execFile` or `spawn` with argument arrays. If a shell is unavoidable, strictly quote variables and validate or escape inputs.

Status

Resolved

[M-06] `chain-connect/src/galachain-utils#generateKeys` - Private keys written with permissive default file modes

Description

Two code paths write private keys without explicitly restricting permissions:

- The `generateKeys` writes keys using shell redirection (`echo '...' > <path>`), inheriting the process `umask`. No explicit `0600` mode is set.
- The `keygen` writes the private key via Node's `writeFile` without `mode: 0600`. Node defaults to `0666` & `umask`, which can leave the file readable on permissive `umasks`.

Impact

The `keygen` writes the private key via Node's `writeFile` without `mode: 0600`. Node defaults to `0666` & `umask`, which can leave the file readable on permissive `umasks`.

Impact

On hosts with a permissive `umask` (commonly seen in CI or developer machines), private key files may be readable by other users.

Recommendation

Replace shell redirections with `fs.writeFileSync(path, data, { mode: 0600 })`.

Status

Resolved

Low

[L-01] `chaincode/src/sales/createTokenSale.ts#createTokenSale` - Missing validation for invalid time range

Description

The `createTokenSale` function validates that start and end times are in the future, but does not validate that end is after start. This allows the creation of token sales with logically invalid time ranges.

Impact

This may lead to broken token sales with invalid time ranges, rendering them non-functional.

Recommendation

Consider adding validation to ensure the end time is after the start time.

Status

Resolved

[L-02] chain-api/src/types/sale.ts#CreateTokenSaleDto - Missing validation decorators on start field

Description

The CreateTokenSaleDto class has inconsistent validation decorators between the start and end fields. The end field has proper validation decorators (@IsOptional(), @Min(0), @IsInt()), but the start field is missing all these validation decorators:

```
@JSONSchema({
  description:
    "Unix timestamp of the date when the sale should end. 0 means that it won't end. "
+
    `By default set to ${CreateTokenSaleDto.DEFAULT_END}.`
})
@IsOptional()
@Min(0)
@IsInt()
public end?: number;

@JSONSchema({
  description:
    "Unix timestamp of the date when the sale should start. 0 means that it starts
immediately. " +
    `By default set to ${CreateTokenSaleDto.DEFAULT_START}.`
})
public start?: number;
```

Impact

Inconsistent validation may cause an invalid start parameter to be accepted.

Recommendation

Consider implementing the validation decorators on the `start` parameter.

Status

Resolved



[L-03] chain-cli#network-up - CLI prints developer private keys in release flow**Description**

The `NetworkUp.run()` calls `printTestAdminCredentials(fabloRoot)`, which prints the dev admin private key to stdout. The helper executes a shell pipeline that `cds` into the network folder and `cats` the private key file.

This call is in the main command flow rather than under a guarded “dev-only” flag.

```
function printTestAdminCredentials(fabloRoot: string): void {  
  
  const commands = [  
  
    `cd "${fabloRoot}"`,  
  
    `echo "Dev admin private key which you can use for signing transactions:"`,  
  
    `echo "  $(cat dev-admin-key/dev-admin.priv.hex.txt)"`  
  
  ];  
  
  execSyncStdio(commands.join(" && "));  
  
}
```

Impact

Leaking the dev admin private key at runtime can lead to immediate signing key compromise wherever this output is captured (terminal logs, CI logs).

Recommendation

Remove this call from default flows (or guard behind an explicit `--print-dev-keys` that is disabled by default in production builds). Also ensure sensitive data is never printed to stdout.

Status

Acknowledged

[L-04] chain-cli#network-up - Input validation mismatch enables the command-injection paths

Description

The `customValidation(flags)` states “Only -, _ and . are allowed”, but the enforced regex does not block backticks (`), does not block spaces, and the error message text itself (“Only - and _ are allowed.”) is inconsistent with the preceding comment (“Only -, _ and . are allowed”).

As a result, dangerous characters can slip into shell-interpolated flags that are later used in the vulnerable calls described in separate issue.

```
function customValidation(flags: any): void {  
  
    const { channel, channelType, chaincodeName, chaincodeDir, envConfig } = flags;  
  
    /*Check if the flags does not have special characters like &, |, ;, :, etc. Only -, _  
    and . and are allowed  
  
    Check the maximum length of the flag is 64 characters */  
  
    const specialChars = /[&\\#,#,+()$~%'"':;?*<>@{}|]/;  
  
    const maxLength = 64;
```

Impact

Validation does not prevent command-injection inputs that exploit chaincode names or paths containing command substitution or quote-breaking payloads.

Recommendation

Fix the validator to allowlist a tight character set for every flag that eventually hits a shell. Validate all flags that are later interpolated into commands (paths, image tags, names).

Status

Resolved

[L-05] chain-client#CAClient - Input validation mismatch enables the command-injection paths

Description

The `enrollUser` uses a global `pendingEnrollments` flag to serialize CA enrollments. The flag is set before the CA call and cleared only on success - exceptions thrown during enroll or wallet write leave the flag set, causing indefinite retries (busy-waiting) for that `userId`:

```
// Verify if the enrollment is already in progress - too many simultaneous
// enrollments will cause the CA to fail with a timeout
if (pendingEnrollments[userId]) {
  // Try again after a short delay
  return new Promise((res) => {
    setTimeout(() => res(undefined), 200);
  }).then(() => enrollUser(caClient, wallet, orgMspId, userId, userSecret));
}
pendingEnrollments[userId] = true;
// Enroll the user, and import the new identity into the wallet.
const enrollment = await caClient.enroll({
  enrollmentID: userId,
  enrollmentSecret: userSecret
});
const x509Identity = {
  credentials: {
    certificate: enrollment.certificate,
    privateKey: enrollment.key.toBytes()
  },
  mspId: orgMspId,
  type: "X.509"
};
await wallet.put(userId, x509Identity);
delete pendingEnrollments[userId];
return x509Identity;
```

```
}
```

There is no `finally` call to clear the semaphore.

Impact

A transient failure locks out that identity's enrollment forever within the process. If this happens to the admin identity, any subsequent client operations that depend on it may stall or spin

Recommendation

Wrap the enrollment block with `try {...} finally { delete pendingEnrollments[userId]; }` so the flag is always cleared. Consider a bounded backoff with a per-ID circuit breaker.

Status

Resolved

QA

[Q-01] Incomplete TODO comments and missing implementations

Description

Multiple TODO comments and incomplete implementations exist throughout the codebase, indicating unfinished work or planned improvements that have not been addressed.

Some examples can be found:

- `chaincode/src/vesting/vestingToken.ts:107`: no validation logic is implemented for vesting token creation.
- `chain-api/src/types/contract.ts:78`: type casting is used instead of proper method overloading.
- `chaincode/src/fees/defineFeeSplitFormula.ts:37`: security validation for fee split formulas is not implemented.

Recommendation

Consider addressing all TODO comments and missing implementations.

Status

Acknowledged

[Q-02] chaincode/src/transfer/transferToken.ts#transferToken - Redundant NFT condition check

Description

The `transferToken` function contains redundant conditional checks for `tokenInstance.isNonFungible`. The same condition is checked twice: once at line 84 and again at line 95:

```
if (tokenInstance.isNonFungible) {  
    fromPersonBalance.removeInstance(tokenInstance.instance, ctx.txUnixTime);  
    toPersonBalance.addInstance(tokenInstance.instance);  
} else {  
    fromPersonBalance.subtractQuantity(quantity, ctx.txUnixTime);  
    toPersonBalance.addQuantity(quantity);  
}  
  
await putChainObject(ctx, fromPersonBalance);  
await putChainObject(ctx, toPersonBalance);@JSONSchema({  
    description:  
        "Unix timestamp of the date when the sale should end. 0 means that it won't end. "  
+  
        `By default set to ${CreateTokenSaleDto.DEFAULT_END}.`  
}))  
  
@IsOptional()  
@Min(0)  
@IsInt()  
public end?: number;  
  
@JSONSchema({  
    description:
```

```
        "Unix timestamp of the date when the sale should start. 0 means that it starts
immediately. " +

        `By default set to ${CreateTokenSaleDto.DEFAULT_START}.`

    })

    public start?: number;

    if (tokenInstance.isNonFungible) {

        tokenInstance.owner = to;

        await putChainObject(ctx, tokenInstance);

    }
```

This creates unnecessary code duplication and reduces maintainability.

Recommendation

Combine both NFT-specific operations (`tokenInstance.isNonFungible` is true) into a single conditional block.

Status

Acknowledged

[Q-03] chain-connect/src/chainApis/FeeApi.ts#FetchFeeAuthorizations

Potentially incorrect method name for FetchFeeAuthorizations

Description

The FetchFeeAuthorizations function uses the method name "DryRun", which is incorrect given the function's name:

```
public FetchFeeAuthorizations(dto: FetchFeeAuthorizationsResponse) {  
  
    return this.connection.submit({  
  
        method: "DryRun",  
  
        payload: dto,  
  
        sign: false,  
  
        url: this.chainCodeUrl,  
  
        requestConstructor: FetchFeeAuthorizationsDto,  
  
        responseConstructor: FetchFeeAuthorizationsResponse  
  
    });  
  
}
```

Recommendation

Consider updating the method name to "FetchFeeAuthorizations".

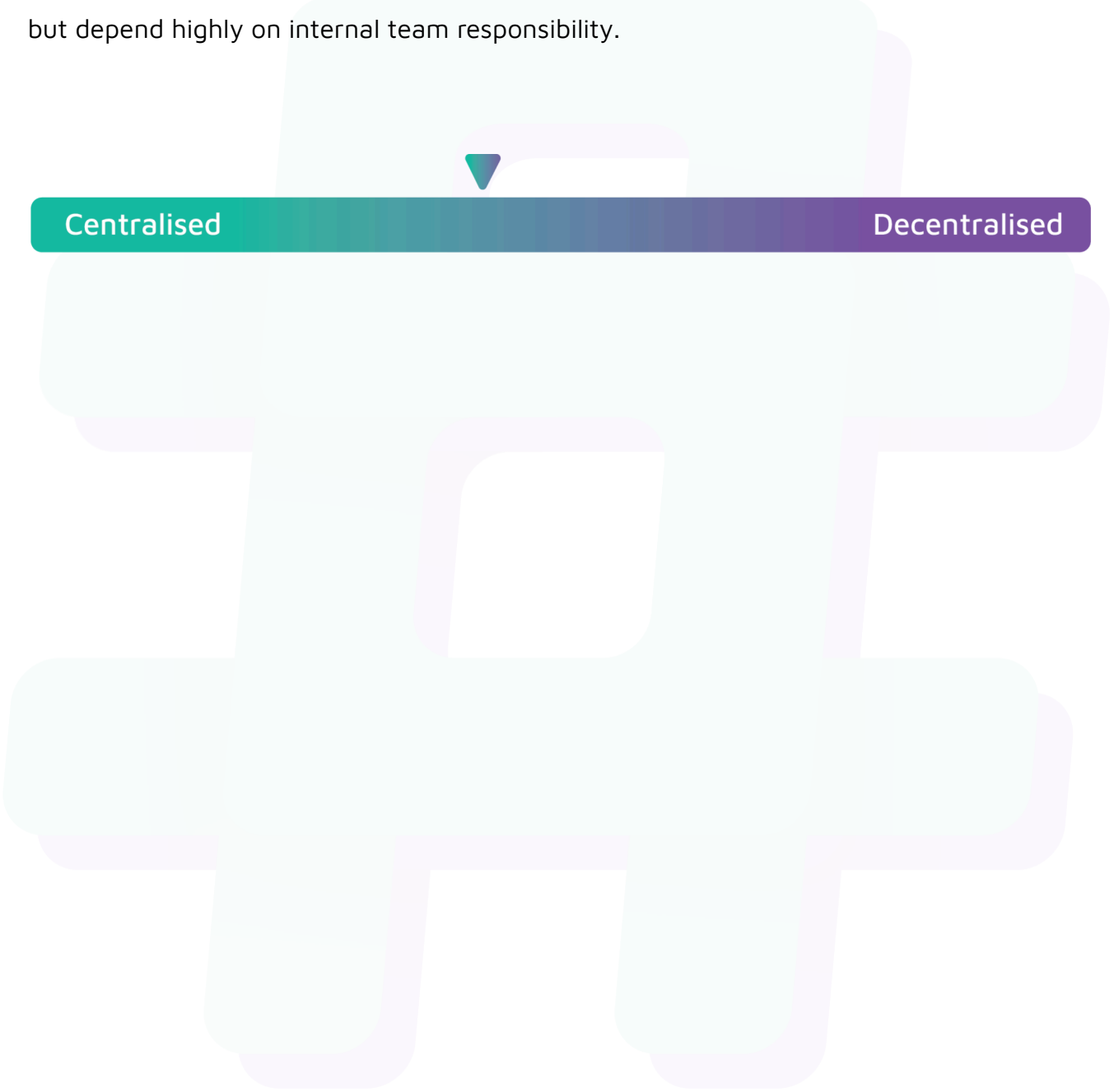
Status

Resolved

Centralisation

The Gala project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Gala project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged."

Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.

